VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



FACULTY OF INFORMATION TECHNOLOGY

LAB REPORT

Lab 01: Xv6 User Programs

Instructor:	Msc. Viet-Long Le	
Leader:	Phuc-Khang Nguyen	24120068
Members:	Minh-Trong Pham	24120151
	Trong-Nghia Hoang	24120103

Ho Chi Minh City, Ngày 16 tháng 10 năm 2026

Table of Contents

1 primes.c **1**

1.1 Tổng quan 1

1.2 Khó khăn và cách xử lý 1

2 tree.c **2**

2.1 Tổng quan 2

2.2 Khó khăn và cách xử lý 3

3 diff.c **5**

3.1 Tổng quan 5

3.2 Khó khăn và Thử thách 6

4 du.c **7**

4.1 Tổng quan 7

4.2 Khó khăn và Thử thách 9

5 pingpong.c **11**

5.1 Tổng quan 11

5.2 Khó khăn và Thử thách 12

6 cp.c **13**

6.1 Tổng quan 13

6.2 Khó khăn và Thử thách 15

7 Bảng phân công nhiệm vụ **16**

1 primes.c

1.1 Tổng quan

Trong chuỗi lọc bằng pipe, mỗi tầng sẽ nhận được các số đã vượt qua các bước lọc số nguyên tố của các tầng trước đó. Vì luồng đầu vào được sắp tăng dần và đã loại các bội số của những số nguyên tố nhỏ hơn, nên giá trị đầu tiên mà một tầng nhận được sẽ là số nguyên tố tiếp theo. Hàm `filter()` dùng giá trị này để lọc luồng còn lại, rồi chuyển các số không chia hết cho nó sang tiến trình con. Quan hệ giữa các tầng được thể hiện trong Hình 1, theo mô hình sàng bằng pipeline của bài thực hành [1].

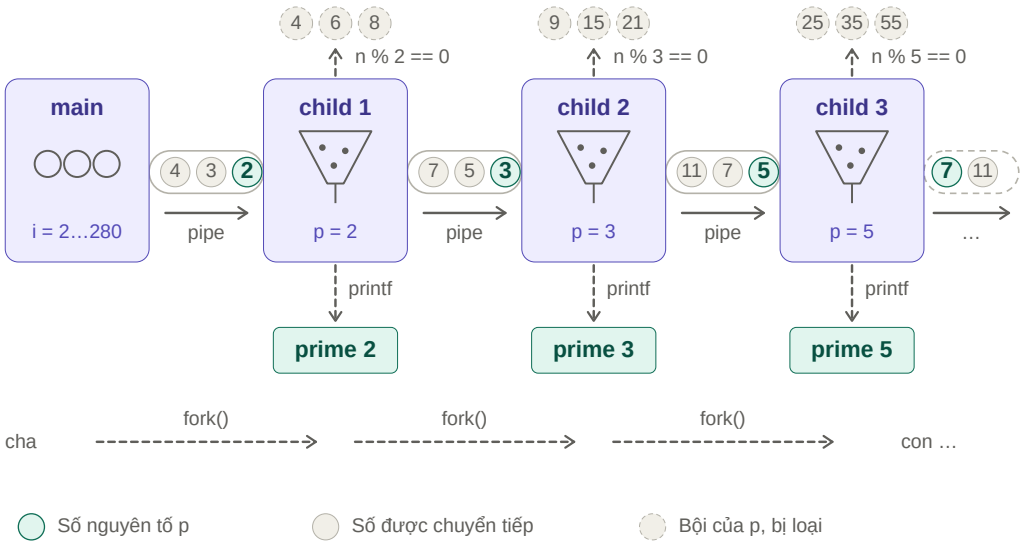


Figure 1: Các tầng lọc và luồng dữ liệu trong `primes.c`.

1.2 Khó khăn và cách xử lý

1. **Tạo bên đọc trước khi truyền dữ liệu:** Bộ đệm pipe của xv6 chỉ có 512 byte, trong khi dãy số đầu vào từ 2 đến 280 chiếm 1116 byte khi truyền bằng số nguyên 4 byte. Nếu ghi toàn bộ dãy số rồi mới tạo tiến trình đọc, bên ghi sẽ bị chặn khi pipe đầy và không thể đi tiếp đến bước tạo bên đọc [2].

   Vì vậy, chương trình gọi `fork()` trước khi gửi dữ liệu để tiến trình con có thể đọc trong lúc cha ghi. Mỗi tầng lọc cũng tạo tầng kế tiếp trước khi chuyển các số còn lại, nhờ đó dữ

liệu được tiêu thụ dọc theo pipeline mà không cần chứa toàn bộ dãy số trong một pipe.

2. **Đóng đúng đầu pipe để kết thúc luồng đọc:** Sau `fork()`, cả cha và con đều kế thừa các đầu pipe. Nếu một đầu ghi không còn dùng vẫn được giữ mở, bên đọc có thể tiếp tục chờ dữ liệu dù bên gửi đã hoàn tất, vì pipe rỗng chỉ báo EOF khi không còn đầu ghi nào mở [2, 3].

   Để tránh tình trạng này, tiến trình con đóng đầu ghi của pipe mới và đầu đọc kế thừa từ tầng trước, còn cha đóng đầu đọc của pipe mới. Khi lọc xong, cha đóng đầu ghi sang con để truyền EOF. Việc loại bỏ các descriptor không còn dùng cũng hạn chế số FD kế thừa qua các tầng, phù hợp với giới hạn `NOFILE = 16` của xv6 [4].

3. **Chờ các tiến trình kết thúc theo quan hệ cha-con:** Một tiến trình chỉ có thể dùng `wait()` để thu hồi con trực tiếp, nên tiến trình chính không thể chờ riêng từng tầng ở sâu trong pipeline. Thay vào đó, mỗi tầng đóng đầu ghi sau khi chuyển hết dữ liệu rồi gọi `wait(0)` để chờ con của mình trước khi thoát [5].

   Trong nhánh kết thúc bình thường, tiến trình cuối chuỗi thoát trước, sau đó các tầng cha lần lượt được thu hồi từ cuối về đầu. Vì mỗi tầng hoàn tất việc chờ con trước khi thoát, lời gọi `wait(0)` ở tiến trình chính chỉ trả về sau khi chuỗi này đã kết thúc.

Giới hạn hiện tại: Các nhánh lỗi chưa áp dụng đầy đủ cách kết thúc trên. Lần đọc đầu của `filter()` còn gộp lỗi đọc với EOF, một số nhánh lỗi thoát trước khi chờ con, và `wait(0)` không kiểm tra mã thoát của con. Do đó, chương trình chưa bảo đảm truyền đầy đủ lỗi ở các tầng về tiến trình chính.

2 tree.c

2.1 Tổng quan

Để có thể đóng thư mục trước khi đi sâu mà vẫn giữ đủ thông tin hiển thị, hàm `tree()` thu thập các mục con vào mảng `children`. Mỗi phần tử lưu tên mục con và cờ thư mục; những mục không được hiển thị sẽ bị loại ngay trong bước thu thập. Sau khi đóng FD của thư mục hiện tại, chương trình duyệt danh sách này để in nhánh và gọi đệ quy cho các thư mục con. Cách tổ chức đó tách việc đọc thư mục khỏi việc duyệt sâu, như Hình 2 minh họa.

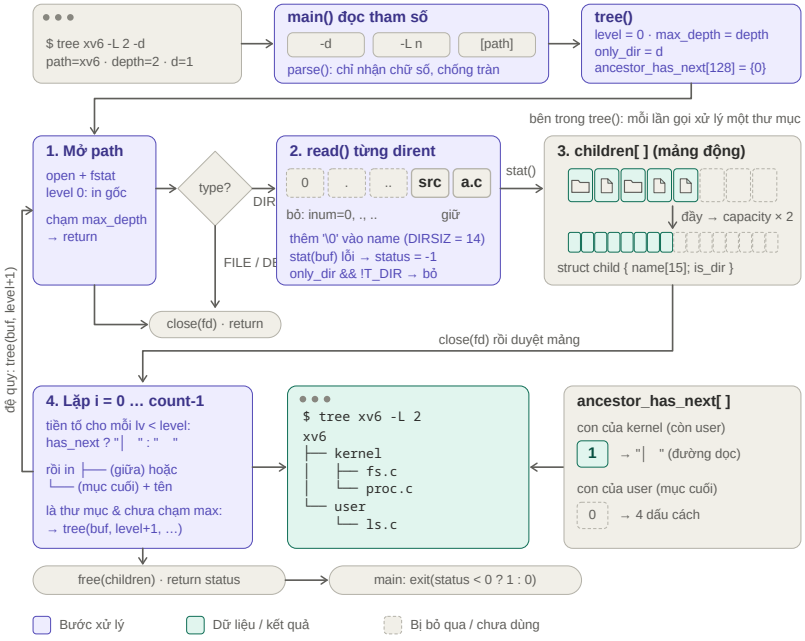


Figure 2: Thu thập mục con, định dạng nhánh và duyệt sâu trong `tree.c`.

2.2 Khó khăn và cách xử lý

1. **Quản lý danh sách mục con và FD:** Nếu để các thư mục cha mở trong suốt quá trình đệ quy, số FD sẽ tăng theo độ sâu và có thể chạm giới hạn `NOFILE = 16` [4]. Vì vậy, chương trình lưu các mục con vào heap rồi đóng FD trước khi gọi đệ quy, giúp số FD mở thêm do thao tác duyệt không tích lũy theo độ sâu cây.

   Danh sách mục con có kích thước thay đổi, nên mảng được cấp phát ban đầu với sức chứa 8 phần tử và tăng gấp đôi khi đầy. Chỉ sau khi cấp phát mới thành công, chương trình mới sao chép dữ liệu và giải phóng mảng cũ; danh sách được thu hồi sau khi duyệt xong thư mục. Nếu cấp phát thất bại, chương trình báo lỗi và thoát với mã 1.

2. **Định dạng nhánh sau khi lọc mục con:** Khi chỉ hiển thị thư mục, mục cuối trong thư mục đang xét có thể là một tệp bị loại khỏi đầu ra. Nếu xác định nhánh cuối trước khi lọc, mục cuối được hiển thị có thể nhận sai ký hiệu nhánh. Vì vậy, chỉ các mục vượt qua bộ lọc mới được đưa vào `children`, rồi `is_last` được tính trên danh sách này.

   Vạch nối dọc còn phụ thuộc vào việc mỗi cấp tổ tiên có mục nào chưa được in hay không. Chương trình lưu trạng thái đó trong `ancestor_has_next` trước khi đi xuống một thư mục

con. Khi in các mức thụt lề, trạng thái còn mục tiếp theo tạo vạch dọc, còn cấp đã kết thúc tạo khoảng trắng; nhánh hiện tại được chọn bằng `is_last`.

3. **Ghép đường dẫn từ tên có độ dài cố định:** Tên trong `struct dirent` có tối đa `DIRSIZ = 14` byte và có thể không chứa ký tự kết thúc chuỗi khi dùng hết độ dài này. Để các thao tác xử lý chuỗi đọc đúng giới hạn, chương trình sao chép tên sang bộ đệm 15 byte rồi thêm byte kết thúc trước khi ghép với đường dẫn cha [6, 3].

   Đường dẫn dài dần khi đi xuống cây, nên trước mỗi lần ghép, chương trình kiểm tra tổng chiều dài đường dẫn cha, dấu phân cách, tên con và byte kết thúc có vừa bộ đệm hay không. Bộ đệm cục bộ `buf[PATHSIZ + 1]` gồm 129 byte, còn đường dẫn truyền cho syscall bị giới hạn bởi `MAXPATH = 128`, kể cả byte kết thúc chuỗi [4]. Mục vượt bộ đệm hoặc không truy xuất được bằng `stat()` sẽ được báo lỗi và bỏ qua, đồng thời lỗi được ghi nhận để quyết định mã thoát.

4. **Ghi nhận và truyền lỗi khi duyệt:** Một mục không truy xuất được không nhất thiết ngăn việc hiển thị các mục khác trong cùng thư mục. Với lỗi `stat()` hoặc đường dẫn quá dài, chương trình đặt `status = -1` rồi tiếp tục các mục còn lại. Nếu `read()` trả về lỗi hoặc một bản ghi thiếu byte, việc thu thập dừng lại, FD được đóng và các mục đã thu thập vẫn được xử lý.

   Lỗi từ lời gọi đệ quy được truyền về cha bằng giá trị âm, nên quá trình duyệt có thể tiếp tục ở các nhánh khác mà mã thoát cuối cùng vẫn phản ánh việc đã có lỗi. Hàm `main()` trả mã 1 khi nhận trạng thái lỗi và mã 0 khi duyệt thành công.

5. **Kiểm tra độ sâu đầu vào và giới hạn đệ quy:** Giá trị độ sâu phải được kiểm tra trước khi dùng để điều khiển việc duyệt. Hàm `parse()` từ chối chuỗi rỗng và ký tự không phải chữ số, đồng thời kiểm tra tràn số trước mỗi lần nhân giá trị tích lũy với 10 rồi cộng chữ số mới. Nhờ đó, giá trị vượt miền số nguyên được báo lỗi thay vì trở thành một giới hạn độ sâu sai.

   Dù các FD đã được đóng trước khi đi sâu, mỗi lời gọi đệ quy vẫn cần user stack cho bộ đệm đường dẫn và các biến cục bộ. Trong cấu hình lab hiện tại, `USERSTACK = 2` tương ứng 8 KiB và không tự mở rộng [4]. Vì vậy, chương trình vẫn có thể tràn stack khi cây quá sâu; độ sâu an toàn phụ thuộc vào khung hàm sau biên dịch và phần stack đã dùng.

3 diff.c

3.1 Tổng quan

`diff.c` thực hiện so sánh đối chiếu hai tệp văn bản theo từng cặp dòng có cùng chỉ số và báo cáo các vị trí có nội dung sai lệch. Khi phát hiện khác biệt, chương trình hiển thị nội dung các dòng tương ứng với tiền tố quy ước (`<` cho tệp thứ nhất và `>` cho tệp thứ hai).

Trong xv6, thư viện người dùng chỉ cung cấp một tập hàm tương đối hạn chế, không có sẵn các hàm đọc dòng như `getline()` hay `fgets()` [3]. Do đó, nhóm xây dựng hàm `readline()` để đọc dữ liệu từ file descriptor bằng `read()`, đồng thời quản lý bộ đệm động và trạng thái ký tự xuống dòng.

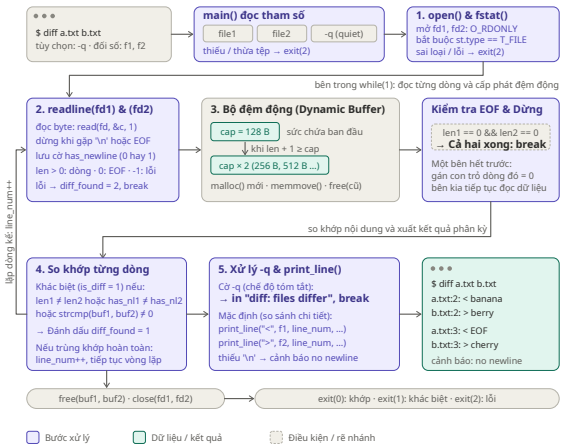


Figure 3: Sơ đồ luồng thực thi và cơ chế so sánh dòng trong `diff.c`

Luồng thực thi gồm các giai đoạn:

- **Mở và kiểm tra tệp:** Phân tích tham số dòng lệnh, mở hai tệp với cờ `O_RDONLY`, sau đó dùng `fstat()` để xác nhận cả hai là tệp thường (`st.type == T_FILE`), tránh xử lý nhầm thư mục hoặc thiết bị.
- **Đọc từng dòng:** Hàm `readline()` sử dụng hai bộ đệm độc lập, mỗi bộ đệm có dung lượng ban đầu 128 byte. Khi một dòng dài hơn sức chứa hiện tại, chương trình cấp phát vùng nhớ mới với dung lượng gấp đôi, sao chép dữ liệu bằng `memmove()` và giải phóng vùng nhớ cũ. Kết quả của `read()` được lưu riêng: `0` là EOF, giá trị âm là lỗi. Nếu lỗi xảy ra ngay từ byte đầu hoặc giữa dòng, `readline()` trả về `-1`, không coi phần dòng đã đọc là dữ liệu hợp lệ. Hàm chỉ trả về `0` khi gặp EOF mà chưa đọc được byte nào; dòng rỗng

có ký tự xuống dòng vẫn có độ dài bằng `1`.

- **So sánh và xuất kết quả:** Với mỗi cặp dòng cùng chỉ số, chương trình đối chiếu chiều dài, cờ `has_newline` và nội dung bằng `strcmp()`. Khi phát hiện khác biệt, hàm `print_line()` hiển thị dòng tương ứng của từng tệp với tiền tố `<` và `>` cùng số dòng để người dùng xác định vị trí sai khác.

3.2 Khó khăn và Thử thách

Trong quá trình triển khai, nhóm tập trung xử lý các trường hợp biên liên quan đến việc đọc dòng và quản lý tài nguyên trên xv6:

1. **Hai tệp có số dòng khác nhau:** Khi một tệp kết thúc trước tệp còn lại, chương trình phải phân biệt được trạng thái hết tệp (EOF) với một dòng rỗng hợp lệ. Nếu chỉ dựa vào độ dài nội dung bằng 0, việc so sánh có thể dừng sớm hoặc bỏ sót những dòng còn lại.
   Giải pháp: Nhóm xử lý riêng trạng thái kết thúc của từng luồng đọc. Hàm `print_line()` cho phép biểu diễn phía đã hết tệp bằng `EOF`, đồng thời tiếp tục đọc và hiển thị những dòng còn lại ở phía kia cho đến khi cả hai luồng đều kết thúc. Cách xử lý này bảo toàn thông tin về các dòng dư, dù không tự động căn chỉnh những đoạn văn bản bị thêm hoặc xóa ở giữa tệp.
2. **Dòng cuối không có ký tự xuống dòng:** Hai dòng có cùng chuỗi ký tự nhưng khác nhau về việc có kết thúc bằng `'\n'` hay không không nên bị xem là hoàn toàn giống nhau. Đây là một trường hợp cần lưu ý khi so sánh tệp văn bản theo dòng [7].
   Giải pháp: Hàm `readline()` lưu cờ `has_newline` cho mỗi dòng và đưa cờ này vào điều kiện so sánh. Nếu dòng cuối không có ký tự xuống dòng, chương trình chủ động xuống dòng khi xuất kết quả và thông báo `no newline at end of file` để phân biệt hai trường hợp.
3. **Chế độ so sánh nhanh và mã thoát:** Khi chỉ cần biết hai tệp có khác nhau hay không, việc in toàn bộ các dòng sai khác gây ra I/O không cần thiết.
   Giải pháp: Khi bật `-q`, chương trình dừng ở khác biệt đầu tiên, đặt `is_diff = 1` và in thông báo `diff: files differ`. Trước khi thoát, chương trình đóng các file descriptor và giải phóng bộ đệm đã cấp phát. Mã thoát được quy ước là `0` nếu hai tệp giống nhau, `1` nếu có khác biệt và `2` nếu xảy ra lỗi, tương thích với quy ước của `diff` trong POSIX [7]. Cụ thể, thiếu hoặc thừa đường dẫn, mở tệp thất bại, lỗi `fstat()`, đầu vào không phải tệp thường, lỗi cấp phát và lỗi đọc đều dẫn đến mã `2`. Trong vòng so sánh, lỗi đặt

`diff_found = 2` rồi dừng, kể cả khi trước đó đã phát hiện khác biệt; các bộ đệm và descriptor được thu hồi trước khi thoát. Với `-q`, chương trình dừng ngay ở khác biệt đầu tiên nên không kiểm tra các lỗi có thể xuất hiện ở phần chưa đọc.

4 du.c

4.1 Tổng quan

`du.c` (*Disk Usage*) tham khảo tiện ích `du` trên các hệ điều hành họ Unix [8], sử dụng thuật toán duyệt đệ quy hậu thứ tự (*post-order bottom-up traversal*) để cộng dồn kích thước tệp trong cây thư mục.

Quy ước tính dung lượng: Phiên bản của nhóm tính theo byte, cộng giá trị `st.size` của các mục `T_FILE` và `T_DEVICE` trong cây con. Chương trình bỏ qua kích thước `st.size` của bản thân mọi thư mục, kể cả thư mục gốc; do đó thư mục rỗng có tổng bằng `0`. Đây là tổng kích thước logic của các tệp, khác với cách thống kê dung lượng khối đĩa của `du` POSIX/GNU, vốn tính cả dung lượng của bản thân thư mục [8]. Quy ước này được áp dụng nhất quán cho đầu ra mặc định và các cờ `-a`, `-s`.

Trong hệ thống tệp xv6 [3], một thư mục được tổ chức như một tập tin đặc biệt chứa danh sách các bản ghi `struct dirent` (16 bytes gồm 2 bytes chỉ mục `inum` và 14 bytes tên `name`). Kích thước byte thực tế `st.size` của từng inode được truy xuất chính xác thông qua lời gọi hệ thống `stat()`.

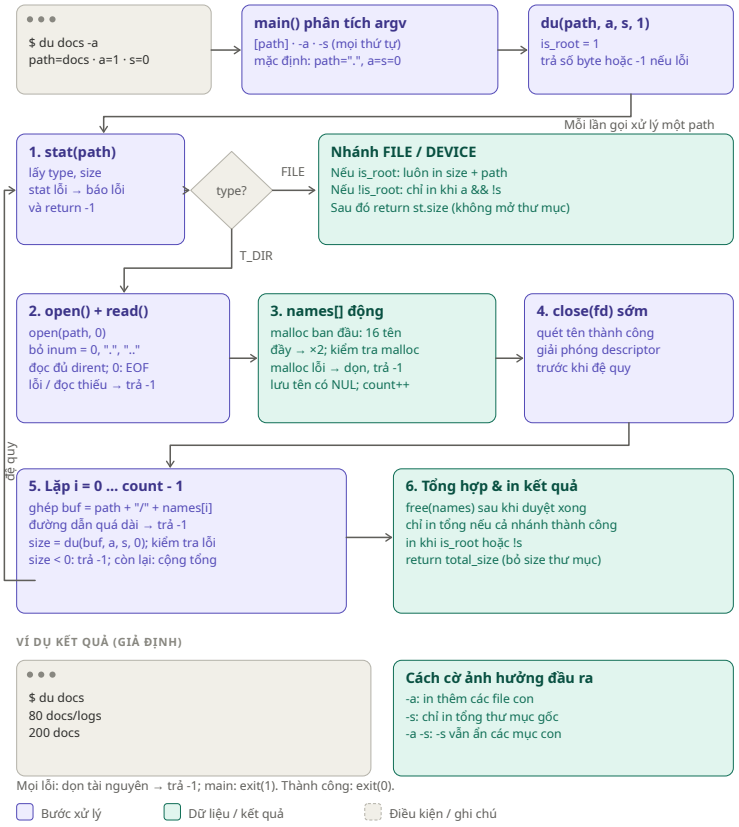


Figure 4: Sơ đồ luồng thực thi duyệt đệ quy Bottom-up và giải phóng FD sớm trong `du.c`

Luồng thực thi diễn ra tuần tự qua các giai đoạn:

- **Khởi tạo và phân tích đối số:** Tiếp nhận tối đa một đường dẫn mục tiêu (mặc định là `"."`), phân tích cờ tóm tắt `-s` (chỉ in kết quả đường dẫn gốc) và cờ chi tiết `-a` (in thêm kích thước tệp con).
- **Điều kiện dừng (Base Case):** Sử dụng `stat(path, &st)`. Nếu đường dẫn trỏ tới một tệp tin thường (`T_FILE`) hoặc tệp thiết bị (`T_DEVICE`), kích thước `st.size` được trả về ngay lập tức; đồng thời in thông tin ra màn hình nếu đường dẫn là gốc được chỉ định hoặc có cờ `-a` (`is_root || (a && !s)`).
- **Duyệt con và cộng dồn (Bottom-up):** Khi gặp thư mục (`T_DIR`), chương trình quét nội dung thư mục, lưu danh sách các tên tệp con hợp lệ, rồi gọi đệ quy `du(buf, a, s, 0)`. Chỉ kết quả không âm mới được cộng vào `total_size`. Sau khi mọi mục con được xử lý thành công, thư mục in tổng nếu là gốc hoặc không bật `-s`, rồi trả tổng về cho cha. Nếu

một mục lỗi, nhánh hiện tại dừng và trả `-1`.

4.2 Khó khăn và Thử thách

Dựa trên kiến trúc của hệ điều hành xv6 và các ca biên trong quản lý tệp tin [3], nhóm nhận thấy các khó khăn như sau:

1. **Giới hạn File Descriptor (`NOFILE = 16`):** Trong kernel xv6, mỗi tiến trình bị giới hạn mở tối đa 16 file descriptors đồng thời (`NOFILE = 16` trong `kernel/param.h`) [3]. Nếu duy trì `fd` của thư mục đang mở trong suốt các lời gọi đệ quy sâu, số FD sẽ tích lũy theo độ sâu, khiến `open()` thất bại khi đạt giới hạn.
   Giải pháp: Đọc toàn bộ tên các mục con vào mảng động `names` trên bộ nhớ heap, sau đó thực hiện lệnh `close(fd)` trước khi bắt đầu bất kỳ lời gọi đệ quy con nào. Nhờ đó, số FD mở thêm do thao tác duyệt duy trì ở mức $O(1)$ (tối đa một FD thư mục trong lúc đọc), không tích lũy theo độ sâu cây.
   Giới hạn còn lại: Mỗi lời gọi `du()` vẫn sử dụng user stack, bao gồm bộ đệm `buf[MAXPATH]` kích thước 512 byte và các biến cục bộ. Trong cấu hình lab hiện tại (`LAB_UTIL`), `USERSTACK = 2` trong `kernel/param.h`, tương ứng 8192 byte (8 KiB); stack này không tự động mở rộng. Vì vậy, chương trình chưa xử lý được cây có độ sâu tùy ý: đệ quy quá sâu vẫn có thể gây tràn user stack. Ngưỡng cụ thể phụ thuộc kích thước khung hàm sau biên dịch và phần stack đã dùng, không chỉ số FD. Khi không bật `LAB_UTIL`, cấu hình trong mã nguồn này dùng `USERSTACK = 1`.
2. **Phòng chống lặp vô hạn và lọc Inode rác:** Mọi thư mục Unix đều chứa hai thực thể liên kết cứng đặc thù là `"."` (thư mục hiện hành) và `".."` (thư mục cha). Ngoài ra, các entry đã xóa trong xv6 vẫn tồn tại trong bảng chỉ mục với `inum == 0`.
   Giải pháp: Bỏ qua các entry có `de.inum == 0` và lọc bỏ `"."` cùng `".."` để không đệ quy quay lại chính thư mục hiện tại hoặc thư mục cha. Điều này ngăn vòng lặp do hai entry đặc biệt gây ra và tránh cộng lặp dung lượng; stack chịu ảnh hưởng khi đệ quy là user stack, vì `du` chạy ở không gian người dùng. Một cây hợp lệ nhưng quá sâu vẫn có thể làm tràn user stack như giới hạn nêu trên.
3. **Quản lý bộ nhớ mảng động và kiểm soát đường dẫn:** Số lượng phần tử trong một thư mục là biến thiên và không thể biết trước, trong khi việc ghép nối chuỗi đường dẫn đệ quy tiềm ẩn nguy cơ tràn mảng đệm.
   Giải pháp: Khai báo `char (*names)[DIRSIZ + 1]`, tức con trỏ tới mảng ký tự có độ dài

cố định. Vùng heap lưu liên tiếp các chuỗi, mỗi phần tử chiếm `DIRSIZ + 1 = 15` byte (gồm byte kết thúc chuỗi), không phải một mảng các con trỏ `char *`. Sức chứa ban đầu là 16 phần tử và tăng gấp đôi (`capacity * 2`) qua `malloc()` và `memmove()` khi danh sách đầy. Cả lần cấp phát đầu và lần mở rộng đều được kiểm tra trước khi sử dụng; nếu thất bại, chương trình báo lỗi, đóng FD, giải phóng mảng cũ nếu có và trả `-1`, không sao chép vào con trỏ rỗng. Trước khi ghép đường dẫn con, chương trình kiểm tra tổng chiều dài đường dẫn cha, dấu phân cách, tên con và byte kết thúc chuỗi không vượt `sizeof(buf)`; mục vượt bộ đệm làm nhánh hiện tại báo lỗi và trả `-1`. Vùng nhớ danh sách được giải phóng sau khi duyệt xong thư mục. Bộ đệm cục bộ 512 byte không mở rộng giới hạn đường dẫn của syscall xv6: `MAXPATH = 128` trong `kernel/param.h` vẫn áp dụng.

4. **Phân biệt tổng hợp lệ với kết quả chưa đầy đủ:** Nếu bỏ qua lỗi đọc hoặc xem mục không truy cập được là có kích thước bằng `0`, chương trình có thể in tổng nhỏ hơn thực tế nhưng vẫn báo thành công.

   Giải pháp: Lưu kết quả `read()` khi đọc thư mục. Chỉ `0` là EOF hợp lệ; giá trị âm hoặc bản ghi ngắn hơn một `struct dirent` đều được báo lỗi. Lỗi `stat()`, `open()`, cấp phát, đọc thư mục hoặc đường dẫn vượt giới hạn được truyền về bằng `-1`. Các tầng cha giải phóng danh sách tên và trả lỗi tiếp, không in tổng của nhánh bị ảnh hưởng. Những dòng của tệp hoặc nhánh đã hoàn tất trước đó vẫn có thể xuất hiện; chương trình không thu hồi đầu ra đã in. Hàm `main()` dùng mã thoát `0` khi tính thành công và `1` khi có lỗi cú pháp hoặc lỗi trong quá trình duyệt.

5 pingpong.c

5.1 Tổng quan

`pingpong.c` minh họa cơ chế giao tiếp liên tiến trình (*Inter-Process Communication* – IPC) và đồng bộ giữa tiến trình cha và tiến trình con trong xv6 [3, 1]. Chương trình thực hiện một lượt trao đổi dữ liệu (*ping-pong*): cha gửi một byte cho con, con nhận rồi gửi lại một byte phản hồi cho cha.

Trong xv6, mỗi pipe là một kênh truyền dữ liệu một chiều dưới dạng luồng byte [3, 2]. Vì vậy, chương trình sử dụng hai pipe độc lập để truyền dữ liệu theo hai hướng: `parent_to_child` (cha ghi, con đọc) và `child_to_parent` (con ghi, cha đọc).

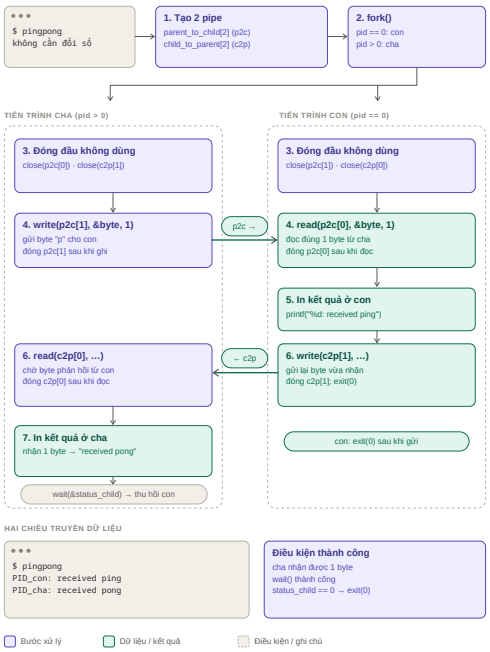


Figure 5: Sơ đồ giao tiếp IPC qua hai pipe và đồng bộ cha – con trong `pingpong.c`

Luồng thực thi gồm các giai đoạn:

- **Khởi tạo kênh truyền:** Tạo hai pipe, sau đó gọi `fork()` để sinh tiến trình con. Hai tiến trình đóng những đầu pipe không sử dụng nhằm tránh giữ file descriptor (FD) không cần thiết (Bảng 1).
- **Trao đổi dữ liệu:** Cha gửi byte `'p'` qua pipe `parent_to_child[1]`. Con đọc dữ liệu từ `parent_to_child[0]`, in dòng `<pid>: received ping`, rồi gửi lại một byte qua

`child_to_parent[1]`. Cha nhận phản hồi từ pipe `child_to_parent[0]` và in `<pid>: received pong`.

- **Thu hồi tài nguyên:** Hai tiến trình đóng các đầu pipe còn sử dụng sau khi hoàn tất I/O. Cha gọi `wait(&status_child)` để chờ và thu hồi tiến trình con, đồng thời kiểm tra kết quả trước khi kết thúc.

Pipe	Cha giữ	Con giữ
`parent_to_child`	Đầu ghi `[1]`	Đầu đọc `[0]`
`child_to_parent`	Đầu đọc `[0]`	Đầu ghi `[1]`

Table 1: Các đầu pipe mỗi tiến trình cần giữ sau `fork()`

5.2 Khó khăn và Thử thách

Trong quá trình hiện thực cơ chế IPC của xv6, nhóm tập trung xử lý các vấn đề về file descriptor, thứ tự trao đổi dữ liệu và quản lý vòng đời tiến trình [3, 9].

1. **Đóng đầu ghi pipe và kiểm soát điều kiện EOF:** Khi pipe rỗng nhưng vẫn còn đầu ghi được mở, `read()` sẽ chờ dữ liệu mới; lệnh chỉ trả về 0 (EOF) khi toàn bộ các đầu ghi đã đóng [3, 2]. Tuy nhiên, với thao tác đọc đúng một byte của chương trình, việc giữ đầu ghi thừa không tự động gây deadlock nếu byte dữ liệu đã đến. Nguy cơ chờ vô hạn xuất hiện khi chương trình tiếp tục đọc trên pipe rỗng để đợi EOF mà vẫn còn một bản sao đầu ghi được giữ mở.

   Giải pháp: Sau `fork()`, con đóng `parent_to_child[1]` và `child_to_parent[0]`; cha đóng `parent_to_child[0]` và `child_to_parent[1]`. Mỗi tiến trình tiếp tục đóng đầu ghi sau khi gửi xong và đóng đầu đọc sau khi nhận xong.
2. **Đảm bảo thứ tự hiển thị giữa hai tiến trình:** Sau `fork()`, lịch chạy của cha và con không cố định. Nếu cho hai tiến trình in kết quả độc lập, thứ tự có thể thay đổi.

   Giải pháp: Sử dụng cơ chế *blocking I/O* của pipe để đồng bộ: con chỉ in `received ping` sau khi nhận byte từ cha, rồi mới gửi phản hồi; cha chỉ in `received pong` sau khi đọc được phản hồi từ con. Vì vậy, thứ tự hai thông báo được xác lập thông qua chính luồng trao đổi dữ liệu, không cần cơ chế khóa bổ sung [3, 1].
3. **Xử lý lỗi khởi tạo và thu hồi tiến trình:** `pipe()` và `fork()` có thể thất bại khi tài nguyên hệ thống không đủ. Trong xv6-riscv, `NPROC = 64` giới hạn số tiến trình, còn `NOFILE = 16` giới hạn số FD mở trên mỗi tiến trình [4]. Tiến trình con đã thoát sẽ ở trạng

thái zombie cho đến khi được thu hồi. Nếu cha thoát trước, xv6 chuyển tiến trình con còn lại cho `init`; đồng thời, `exit()` tự đóng các FD đang mở [3, 5].

Giải pháp: Kiểm tra kết quả mỗi lời gọi `pipe()` và `fork()`. Nếu khởi tạo thất bại, chương trình đóng các FD đã tạo rồi thoát với mã lỗi. Trong luồng thành công, cha gọi `wait()` để thu hồi con, tránh để zombie tồn tại trong thời gian cha còn hoạt động.

6 cp.c

6.1 Tổng quan

`cp.c` hiện thực chức năng sao chép nội dung của một tệp thông thường sang một đường dẫn đích trong xv6, tham khảo tiện ích `cp` của POSIX [10].

Trong xv6, thông tin tệp được quản lý bằng inode; cặp giá trị `(dev, ino)` giúp xác định hai đường dẫn có đang tham chiếu đến cùng một inode hay không [3]. Chương trình đọc dữ liệu từ tệp nguồn được mở bằng `O_RDONLY` và ghi sang tệp đích bằng tổ hợp cờ `O_WRONLY` | `O_CREATE` | `O_TRUNC`, thông qua bộ đệm có kích thước cố định.

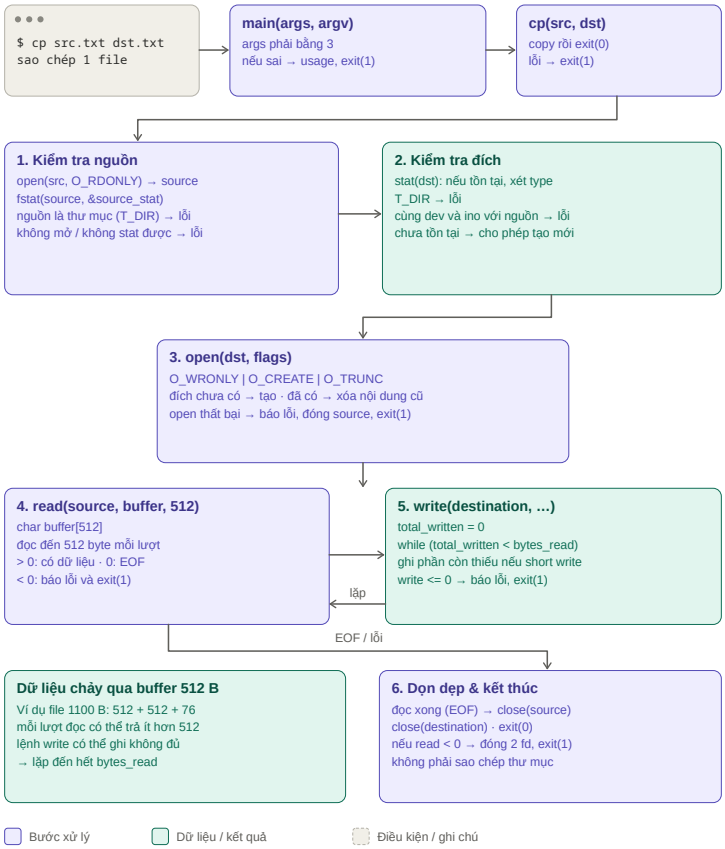


Figure 6: Sơ đồ kiểm tra inode và sao chép dữ liệu theo khối trong `cp.c`

Luồng thực thi gồm các giai đoạn:

- **Kiểm tra đối số và tệp:** Yêu cầu đúng hai đối số đường dẫn (`argc == 3`), mở tệp nguồn bằng `O_RDONLY` và dùng `fstat()` để từ chối thư mục. Nếu đích đã tồn tại, dùng `stat()` kiểm tra kiểu tệp và so sánh cặp `dev`, `ino` nhằm phát hiện trường hợp trùng tệp.
- **Mở đích và chuẩn bị bộ đệm:** Sau khi kiểm tra, mở tệp đích với tổ hợp các cờ `O_WRONLY` | `O_CREATE` | `O_TRUNC`. Sử dụng bộ đệm `buffer[512]` để truyền dữ liệu theo từng khối.
- **Sao chép dữ liệu:** Lặp lại việc `read()` tối đa 512 byte từ nguồn, rồi ghi dữ liệu đã đọc sang đích. Vòng lặp ghi kiểm tra số byte được ghi và tiếp tục nếu còn dữ liệu chưa ghi hết. Khi `read()` trả về 0 (EOF), chương trình đóng các FD và kết thúc thành công; nếu I/O trả về lỗi, chương trình chuyển sang nhánh xử lý lỗi.

6.2 Khó khăn và Thử thách

Trong quá trình xây dựng thao tác sao chép tệp trên xv6, nhóm chú trọng việc tránh ghi đè nhầm tệp nguồn, kiểm tra kết quả I/O và dọn dẹp tài nguyên [3].

1. **Ngăn việc ghi đè vào chính tệp nguồn:** Nếu nguồn và đích trỏ đến cùng một inode, thao tác mở đích với `O_TRUNC` sẽ cắt kích thước của tệp thường về 0, khiến dữ liệu nguồn bị mất trước khi sao chép [11].
   Giải pháp: Trước khi mở đích, chương trình dùng `stat()` để đối chiếu: `source_stat.dev` == `destination_stat.dev` cùng với `source_stat.ino` == `destination_stat.ino`. Nếu cả hai điều kiện trên đều thỏa mãn, chương trình thông báo lỗi và đóng FD nguồn mà không thực hiện `O_TRUNC`.
2. **Kiểm tra kết quả ghi dữ liệu:** Trong giao diện I/O kiểu Unix, mã trả về của `write()` cần được kiểm tra thay vì giả định mọi lần ghi đều hoàn tất. Riêng trên xv6, đường ghi của tệp thông thường xử lý nhiều khối trong kernel và thường trả về đủ số byte yêu cầu hoặc mã lỗi [12].
   Giải pháp: Dùng biến `total_written` để theo dõi lượng dữ liệu đã ghi. Nếu `write()` trả về số byte dương nhỏ hơn số byte cần ghi, chương trình tiếp tục ghi tiếp từ vị trí `buffer` + `total_written`; nếu trả về giá trị không dương, chương trình xử lý như lỗi ghi.
3. **Từ chối sao chép thư mục và dọn dẹp FD:** Thư mục trong xv6 chứa dữ liệu cấu trúc hệ thống tệp. Việc sao chép thô các byte này sang tệp thường không tương đương với sao chép cây thư mục [3].
   Giải pháp: Dùng `fstat()` hoặc `stat()` để từ chối nguồn hoặc đích có kiểu `T_DIR`. Khi `open()`, `read()` hoặc `write()` thất bại, chương trình đóng các FD đã mở và trả về mã lỗi, thay vì tiếp tục với dữ liệu chưa được sao chép đầy đủ.

7 Bảng phân công nhiệm vụ

Bảng 2 tổng kết chi tiết phân công công việc, nhiệm vụ triển khai mã nguồn, xây dựng báo cáo và mức độ đóng góp của từng thành viên trong nhóm đối với Bài thực hành 01.

Họ và tên	**MSSV & Vai trò**	**Nhiệm vụ đảm nhiệm**	**Đóng góp**
Nguyễn Phúc Khang	24120068 (Nhóm trưởng)	• Hiện thực chương trình `primes.c` (sàng nguyên tố qua pipeline, đồng bộ cascade wait). • Hiện thực chương trình `tree.c` (duyệt cây thư mục DFS, hỗ trợ cờ `-L`, `-d`). • Soạn thảo nội dung báo cáo mục `primes.c` và `tree.c`. • Quản trị kho mã nguồn Git, thiết lập linter và đóng gói phát hành bài nộp.	100%
Hoàng Trọng Nghĩa	24120103 (Thành viên)	• Hiện thực chương trình `diff.c` (so sánh tệp theo dòng, hỗ trợ cờ `-q`). • Hiện thực chương trình `du.c` (tính dung lượng bottom-up, tối ưu $O(1)$ FD, hỗ trợ cờ `-a`, `-s`). • Soạn thảo nội dung báo cáo mục `diff.c` và `du.c`. • Kiểm thử các trường hợp biên và xử lý lỗi hệ thống.	100%
Phạm Minh Trọng	24120151 (Thành viên)	• Hiện thực chương trình `pingpong.c` (giao tiếp IPC hai chiều qua hai pipe). • Hiện thực chương trình `cp.c` (sao chép tệp theo khối, kiểm tra trùng inode). • Soạn thảo nội dung báo cáo mục `pingpong.c` và `cp.c`. • Thiết kế sơ đồ kiến trúc và luồng dữ liệu (workflow diagrams).	100%

Table 2: Bảng phân công nhiệm vụ và mức độ hoàn thành của các thành viên trong nhóm

References

[1] MIT CSAIL PDOS Group, "Lab: xv6 and Unix utilities," MIT 6.1810: Operating System Engineering, https://pdos.csail.mit.edu/6.1810/2024/labs/util.html, 2024, section: primes; accessed 2026-10-16.

[2] MIT PDOS, "xv6-riscv source code: kernel/pipe.c," GitHub repository, https://github.com/mit-pdos/xv6-riscv/blob/riscv/kernel/pipe.c, implements pipe read, write, and close; branch riscv; accessed 2026-10-16.

[3] R. Cox, F. Kaashoek, and R. Morris, *xv6: A Simple, Unix-like Teaching Operating System*, 2024, rISC-V edition, revision 4 (August 31, 2024). https://pdos.csail.mit.edu/6.828/2024/xv6/book-riscv-rev4.pdf.

[4] MIT PDOS, "xv6-riscv source code: kernel/param.h," GitHub repository, https://github.com/mit-pdos/xv6-riscv/blob/riscv/kernel/param.h, defines NPROC, NOFILE, and MAXPATH; branch riscv; accessed 2026-10-16.

[5] ——, "xv6-riscv source code: kernel/proc.c," GitHub repository, https://github.com/mit-pdos/xv6-riscv/blob/riscv/kernel/proc.c, process exit, reparenting, and wait; branch riscv; accessed 2026-10-16.

[6] ——, "xv6-riscv source code: kernel/fs.h," GitHub repository, https://github.com/mit-pdos/xv6-riscv/blob/riscv/kernel/fs.h, defines DIRSIZ and struct dirent; branch riscv; accessed 2026-10-16.

[7] IEEE and The Open Group, *diff: Compare Two Files*, issue 7, 2018 edition ed., The Open Group Base Specifications, 2018, iEEE Std 1003.1-2017. https://pubs.opengroup.org/onlinepubs/9699919799/utilities/diff.html.

[8] ——, *du: Estimate File Space Usage*, issue 7, 2018 edition ed., The Open Group Base Specifications, 2018, iEEE Std 1003.1-2017. https://pubs.opengroup.org/onlinepubs/9699919799/utilities/du.html.

[9] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018, version 1.00 (August 2018). Free PDF: https://pages.cs.wisc.edu/~remzi/OSTEP/.

[10] IEEE and The Open Group, *cp: Copy Files*, issue 7, 2018 edition ed., The Open Group Base Specifications, 2018, iEEE Std 1003.1-2017. https://pubs.opengroup.org/onlinepubs/9699919799/utilities/cp.html.

[11] MIT PDOS, "xv6-riscv source code: kernel/sysfile.c," GitHub repository, https://github.com/mit-pdos/xv6-riscv/blob/riscv/kernel/sysfile.c, open system call and O_TRUNC behavior; branch riscv; accessed 2026-10-16.

[12] ——, "xv6-riscv source code: kernel/file.c," GitHub repository, https://github.com/mit-pdos/xv6-riscv/blob/riscv/kernel/file.c, file descriptor layer and filewrite; branch riscv; accessed 2026-10-16.